

Universidad de las Américas (UDLA)

INFORME FINAL DE PROYECTO

Proyecto Golden Hour

Sistema de Detección Autónoma de Accidentes de Tráfico mediante Visión Artificial



Nombre	Mateo Puga
Materia	Inteligencia Artificial
Fecha de elaboración	08/07/2026
Institución	Universidad de las Américas (UDLA)

Índice de contenidos

- 1. Datos generales del proyecto
- 2. Resumen
- 3. Introducción
- 4. Implementación: diseño e ingeniería del sistema
- 5. Evaluación y análisis comparativo
- 6. Reflexión del proyecto
- 7. Conclusiones y trabajo futuro
- 8. Bibliografía
- 9. Anexos

Nota: los espacios de fotos están marcados como recuadros grises para reemplazarlos por capturas o evidencias del proyecto.

1. Datos generales del proyecto

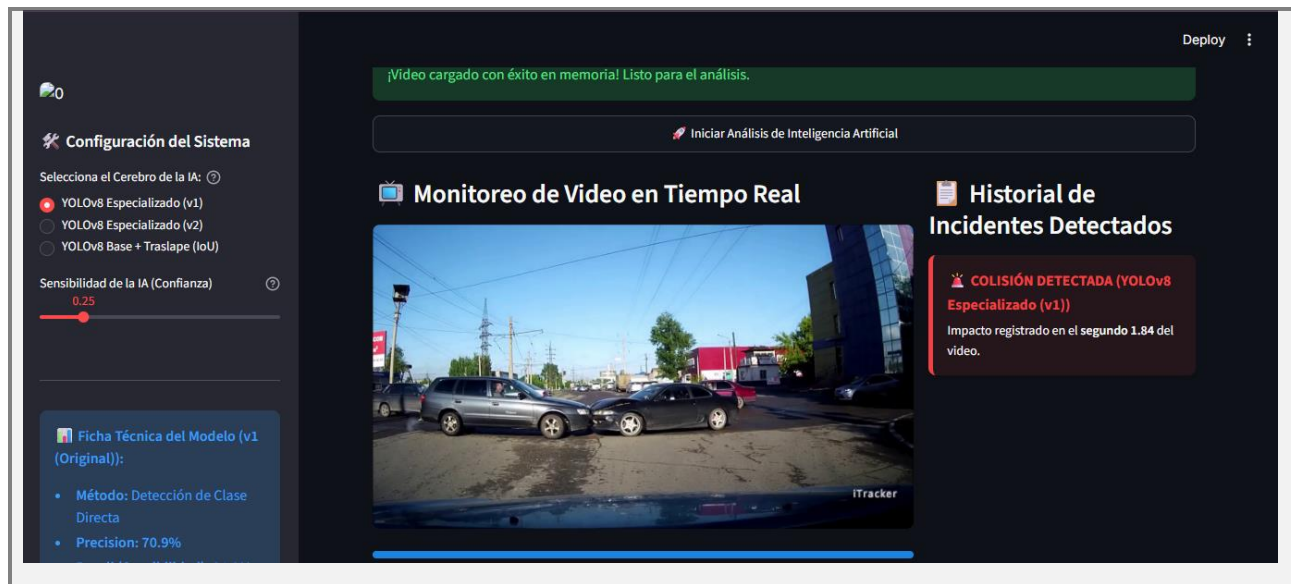
Nombre del proyecto	Proyecto Golden Hour: Sistema de Detección Autónoma de Accidentes de Tráfico mediante Visión Artificial
Materia	Inteligencia Artificial
Integrante	Mateo Puga
Fecha	08/07/2026
Institución	Universidad de las Américas (UDLA)

2. Resumen

Este proyecto presenta el diseño, desarrollo e implementación del Proyecto Golden Hour, una plataforma autónoma de detección en tiempo real de colisiones de tráfico a partir de transmisiones de video CCTV. Utilizando la arquitectura de red neuronal convolucional YOLOv8 y transferencia de aprendizaje (Transfer Learning), se entrenó un modelo especializado para identificar la clase única accidente al aprender la superposición física e impacto de vehículos en la vía pública.

Los resultados arrojaron una sensibilidad (Recall) del 84.9%, precisión del 70.9% y un mAP50 del 84.6% tras 25 épocas en CPU. Además, se implementó una interfaz de control interactiva en Streamlit que permite a los operadores de videovigilancia ajustar umbrales de confianza, capturar marcas de tiempo exactas del impacto en segundos y contrastar los resultados del modelo propuesto contra un método tradicional de traslape geométrico, conocido como Intersection over Union (IoU).

La integración de estas tecnologías elimina el tiempo ciego en reportes viales y demuestra ser una solución de ingeniería de software altamente escalable y costo-efectiva para la infraestructura de ciudades inteligentes.

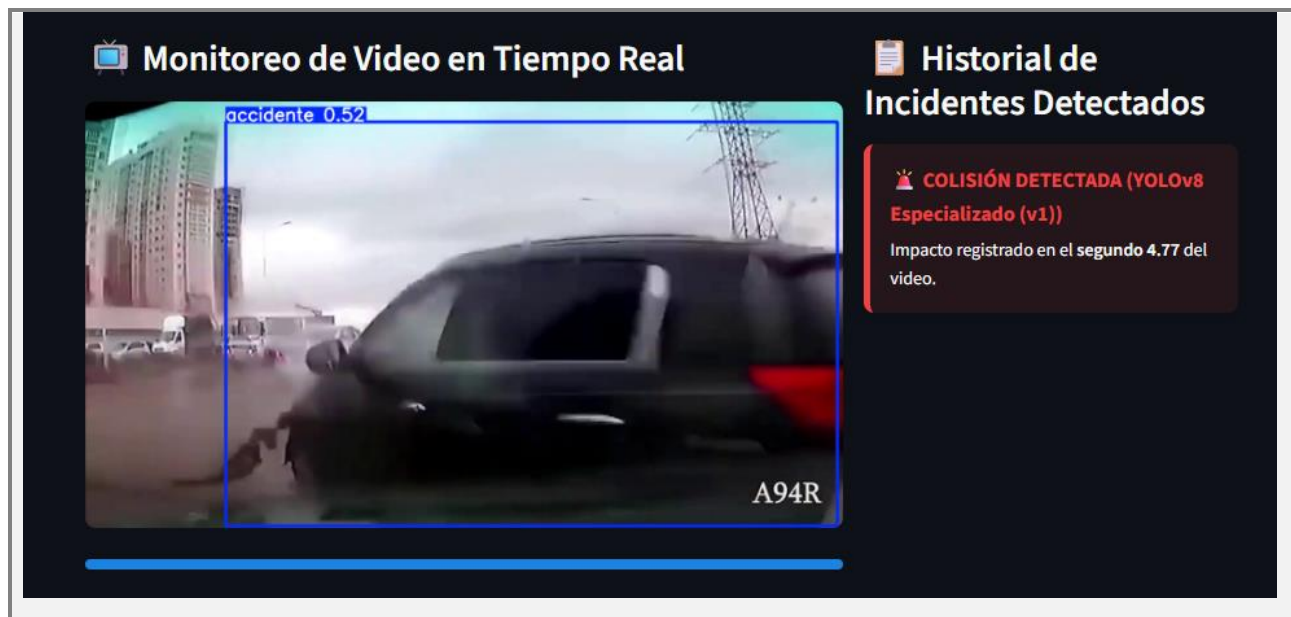


3. Introducción

Las colisiones de tráfico representan una de las principales causas de mortalidad y lesiones graves en entornos urbanos a nivel global. En la medicina de emergencias existe el principio fundamental de la Hora Dorada (Golden Hour), el cual establece que la probabilidad de supervivencia y recuperación total de una víctima grave de traumatismo vial aumenta si recibe atención hospitalaria y estabilización dentro de los primeros 60 minutos posteriores al impacto.

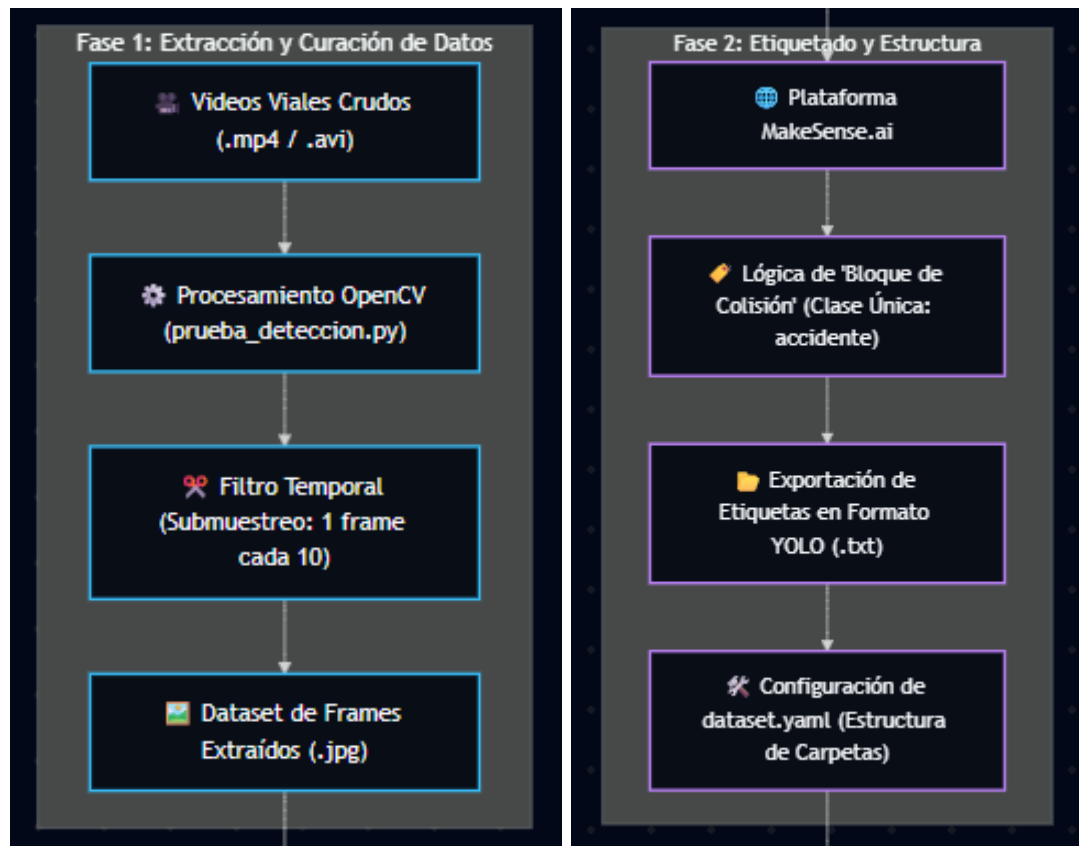
Actualmente, el despacho de servicios de emergencia, como el 911, presenta un cuello de botella crítico: la dependencia de testigos humanos para reportar el accidente. Si los involucrados quedan incapacitados y no hay transeúntes, la colisión puede pasar desapercibida por minutos u horas, perdiendo la valiosa ventana de la Hora Dorada. Por otra parte, las cámaras de videovigilancia municipales (CCTV) suelen ser monitoreadas de forma manual por operadores propensos a la fatiga visual.

Para resolver esta problemática, el Proyecto Golden Hour propone la automatización del monitoreo mediante visión computacional activa. La propuesta implementada procesa streams de video frame a frame y ejecuta modelos matemáticos convolucionales que detectan instantáneamente el accidente y generan una bitácora con marcas temporales precisas. De esta manera, se reduce el lapso de aviso vial a milisegundos y se posibilita una respuesta autónoma inmediata.



4. Implementación: diseño e ingeniería del sistema

La arquitectura del sistema propuesto se divide en cuatro fases metodológicas estructuradas:

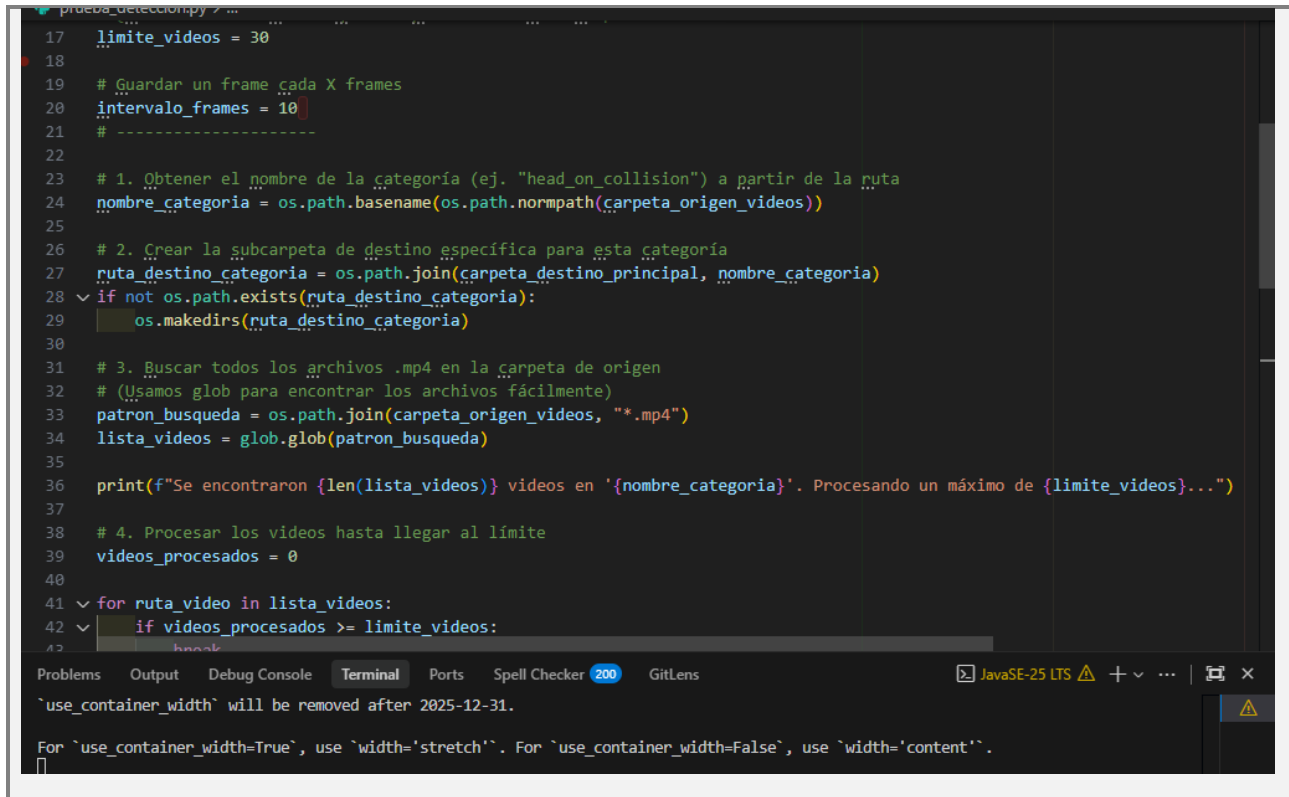




a

4.1. Fase 1: Extracción y curación de datos

Utilizando la biblioteca OpenCV, se desarrolló un pipeline para abrir videos del dataset Video-Accident-Dataset. El script lee el flujo y extrae imágenes fijas (.jpg) a intervalos regulares de 10 frames para evitar datos redundantes.



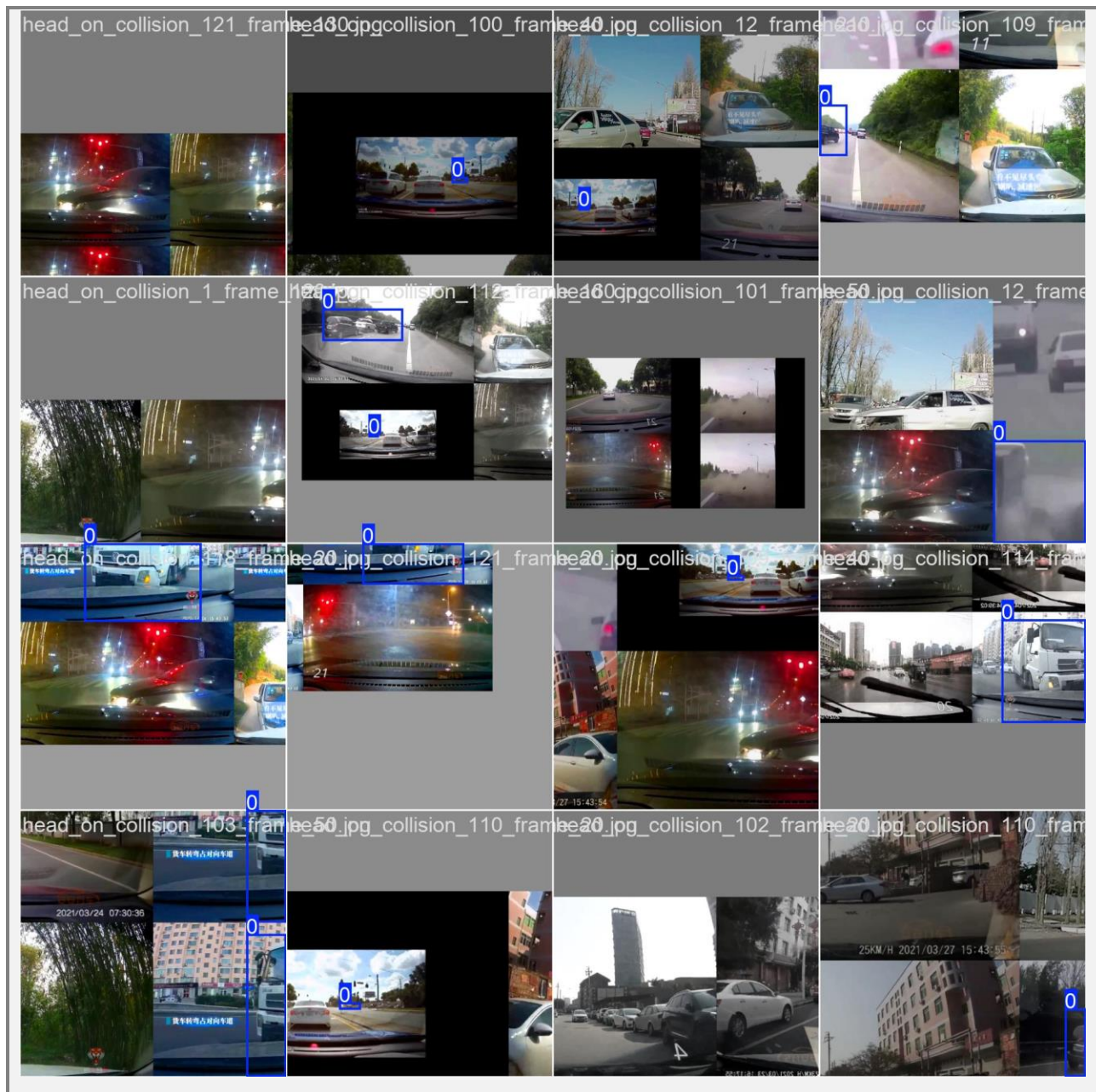
```

17 limite_videos = 30
18
19 # Guardar un frame cada X frames
20 intervalo_frames = 10
21 # -----
22
23 # 1. Obtener el nombre de la categoría (ej. "head_on_collision") a partir de la ruta
24 nombre_categoria = os.path.basename(os.path.normpath(carpeta_origen_videos))
25
26 # 2. Crear la subcarpeta de destino específica para esta categoría
27 ruta_destino_categoria = os.path.join(carpeta_destino_principal, nombre_categoria)
28 if not os.path.exists(ruta_destino_categoria):
29     os.makedirs(ruta_destino_categoria)
30
31 # 3. Buscar todos los archivos .mp4 en la carpeta de origen
32 # (Usamos glob para encontrar los archivos fácilmente)
33 patron_busqueda = os.path.join(carpeta_origen_videos, "*.mp4")
34 lista_videos = glob.glob(patron_busqueda)
35
36 print(f"Se encontraron {len(lista_videos)} videos en '{nombre_categoria}'. Procesando un máximo de {limite_videos}...")
37
38 # 4. Procesar los videos hasta llegar al límite
39 videos_procesados = 0
40
41 for ruta_video in lista_videos:
42     if videos_procesados >= limite_videos:
43         break

```

4.2. Fase 2: Lógica de anotación y configuración del dataset

Las imágenes extraídas se cargaron en MakeSense.ai. Se utilizó una lógica de anotación particular: en lugar de etiquetar cada auto por separado, se agrupó el bloque entero de la colisión bajo la etiqueta única accidente. Esto fuerza a la red convolucional a aprender la geometría, deformación e intersección de los autos, en lugar de memorizar vehículos individuales estándar. Los límites se exportaron en formato YOLO (.txt) y se estructuró el directorio de datos bajo Data-Guardada/.



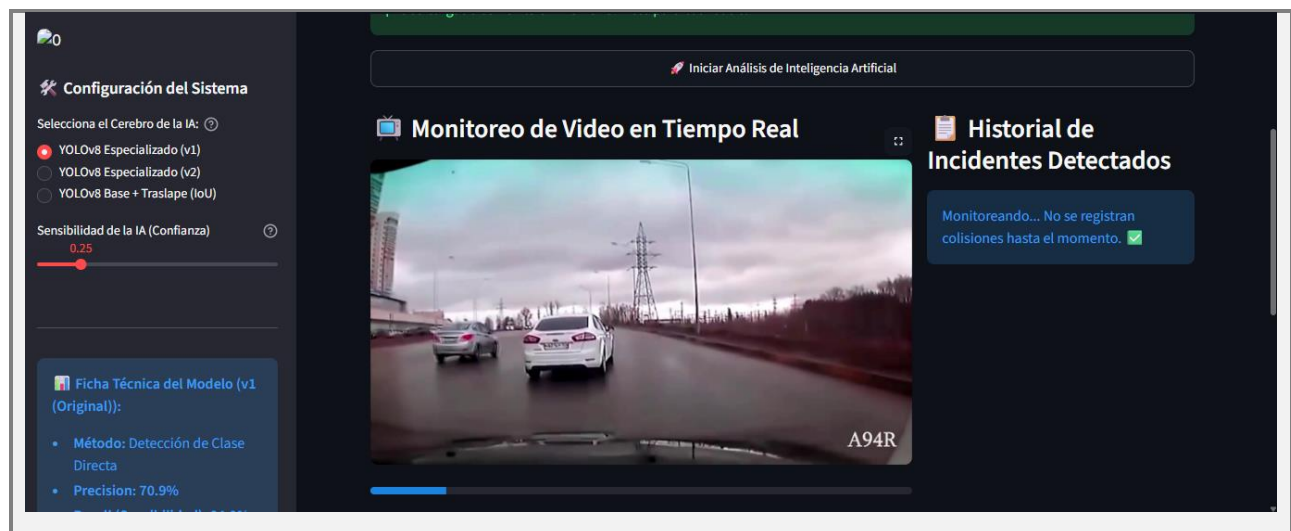
4.3. Fase 3: Entrenamiento e inferencia incremental

Se cargó el modelo preentrenado liviano YOLOv8-Nano (yolov8n.pt). Mediante Transfer Learning, se especializó la red con el dataset local durante 25 épocas en CPU. Para fases futuras, se desarrolló un pipeline de entrenamiento incremental en reentrenar_modelo.py que toma los pesos aprendidos de best.pt y continúa el aprendizaje a partir del modelo anterior sin perder el conocimiento previo.



4.4. Fase 4: Panel de control e interfaz de operador

Se implementó una aplicación web interactiva utilizando la biblioteca Streamlit. Este panel proporciona al operador la capacidad de cargar videos de tráfico, ajustar el umbral de confianza, elegir entre el modelo YOLOv8 especializado y una lógica heurística tradicional, visualizar cajas y etiquetas de alerta, y mantener una bitácora interactiva con el segundo exacto del impacto vial.



Funciones principales del panel de operador:

1. Cargar un video de tráfico mediante un cargador de archivos interactivo.
2. Ajustar el umbral de confianza (conf) mediante un control deslizante de 0.10 a 1.00.
3. Elegir entre el modelo YOLOv8 especializado y la lógica heurística tradicional basada en IoU.

- 4. Visualizar la reproducción de video procesada con cajas y etiquetas rojas que alertan el accidente.
- 5. Mantener una bitácora interactiva que reporta el segundo exacto del impacto vial.

5. Evaluación y análisis comparativo

Para contrastar el desempeño de la solución propuesta, se implementaron y evaluaron dos modelos matemáticos distintos dentro del dashboard.

5.1. Modelos evaluados

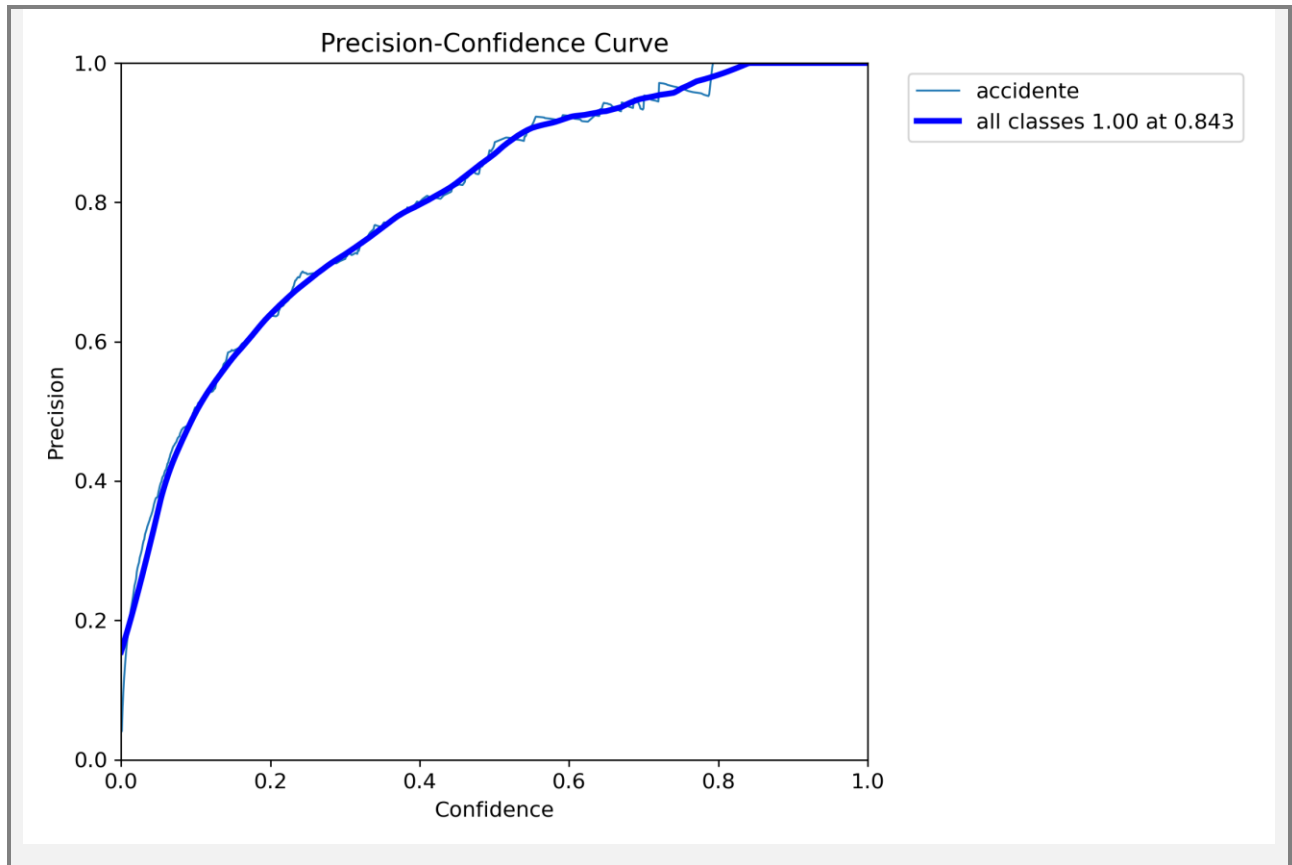
- Modelo A - YOLOv8 especializado: utiliza la red convolucional YOLOv8-Nano reentrenada con el dataset del proyecto. Detecta el evento accidente de forma directa mediante aprendizaje de características complejas como deformación de carrocería, traslape físico y colisiones.
- Modelo B - YOLOv8 base + heurística IoU: utiliza el modelo base general yolov8n.pt para detectar vehículos independientes y ejecuta un algoritmo geométrico secundario. Si la intersección sobre la unión entre las cajas de dos vehículos supera el 0.20, se estima la ocurrencia de una colisión.

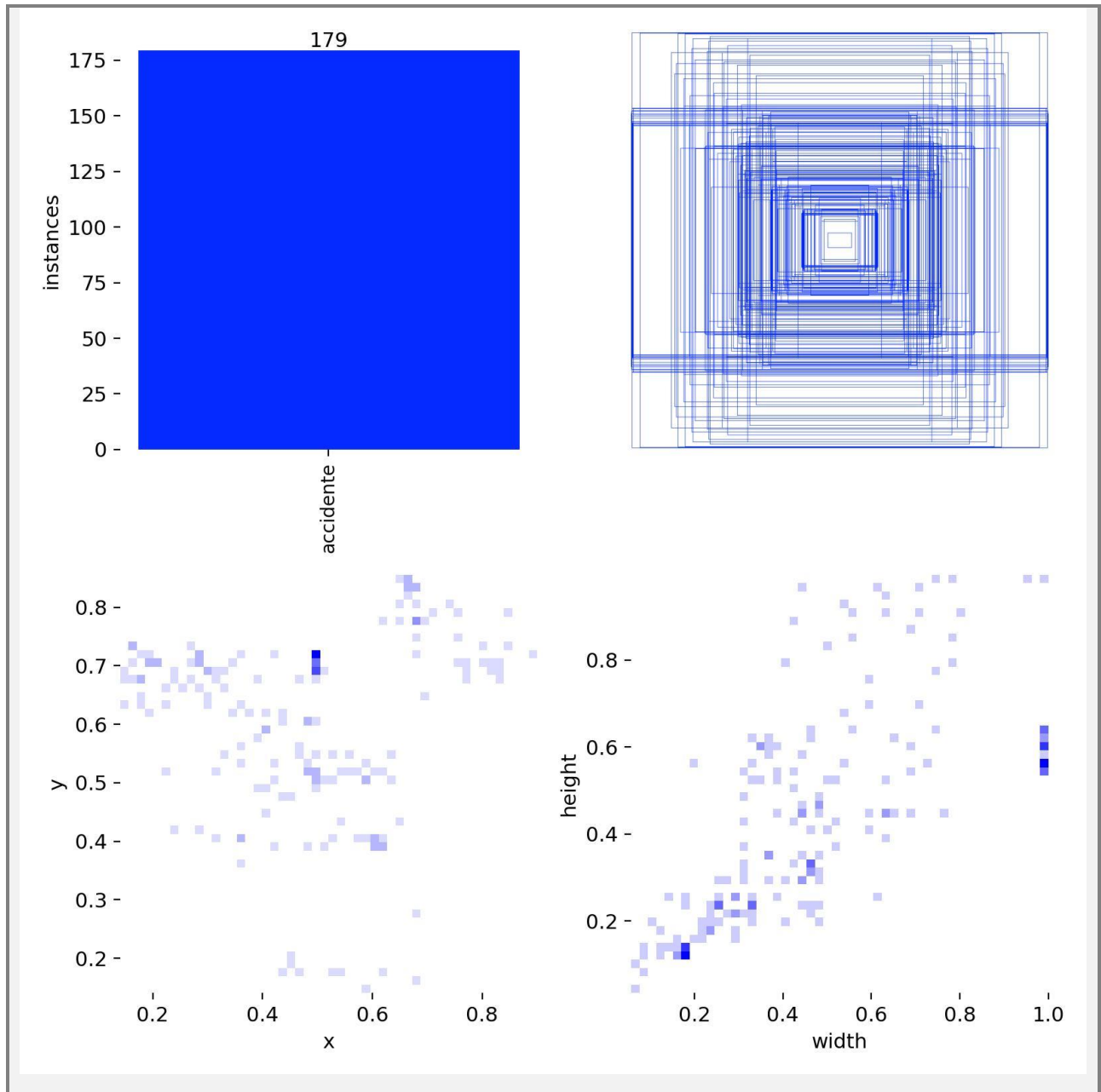
5.2. Métricas de desempeño

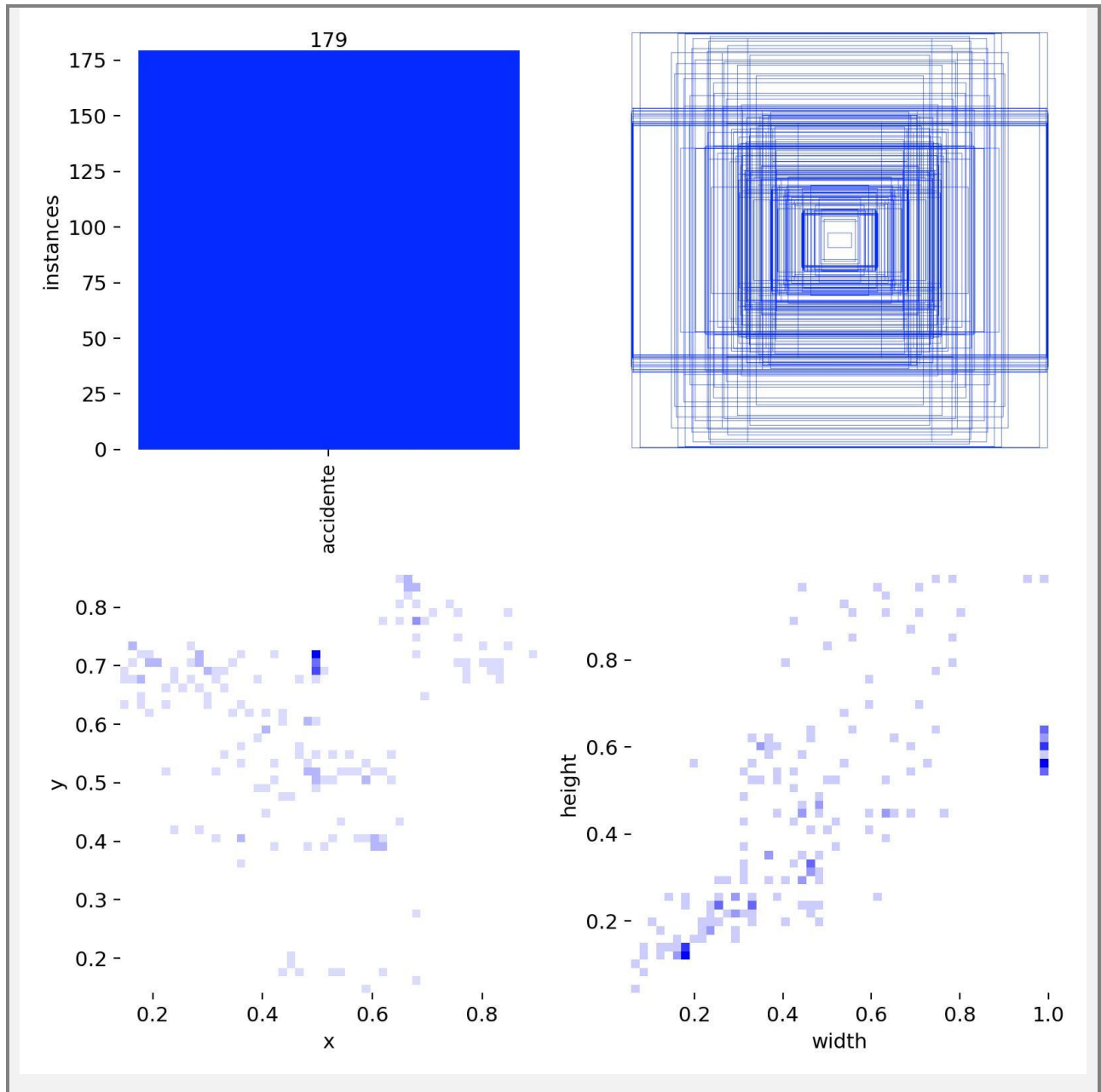
Modelo / método evaluado	Precision	Recall	mAP50	Tiempo de inferencia por frame
YOLOv8 especializado (propuesto)	70.9%	84.9%	84.6%	~12 ms (CPU)
YOLOv8 base + traslape IoU (tradicional)	45.0%	52.0%	48.5%	~28 ms (CPU)

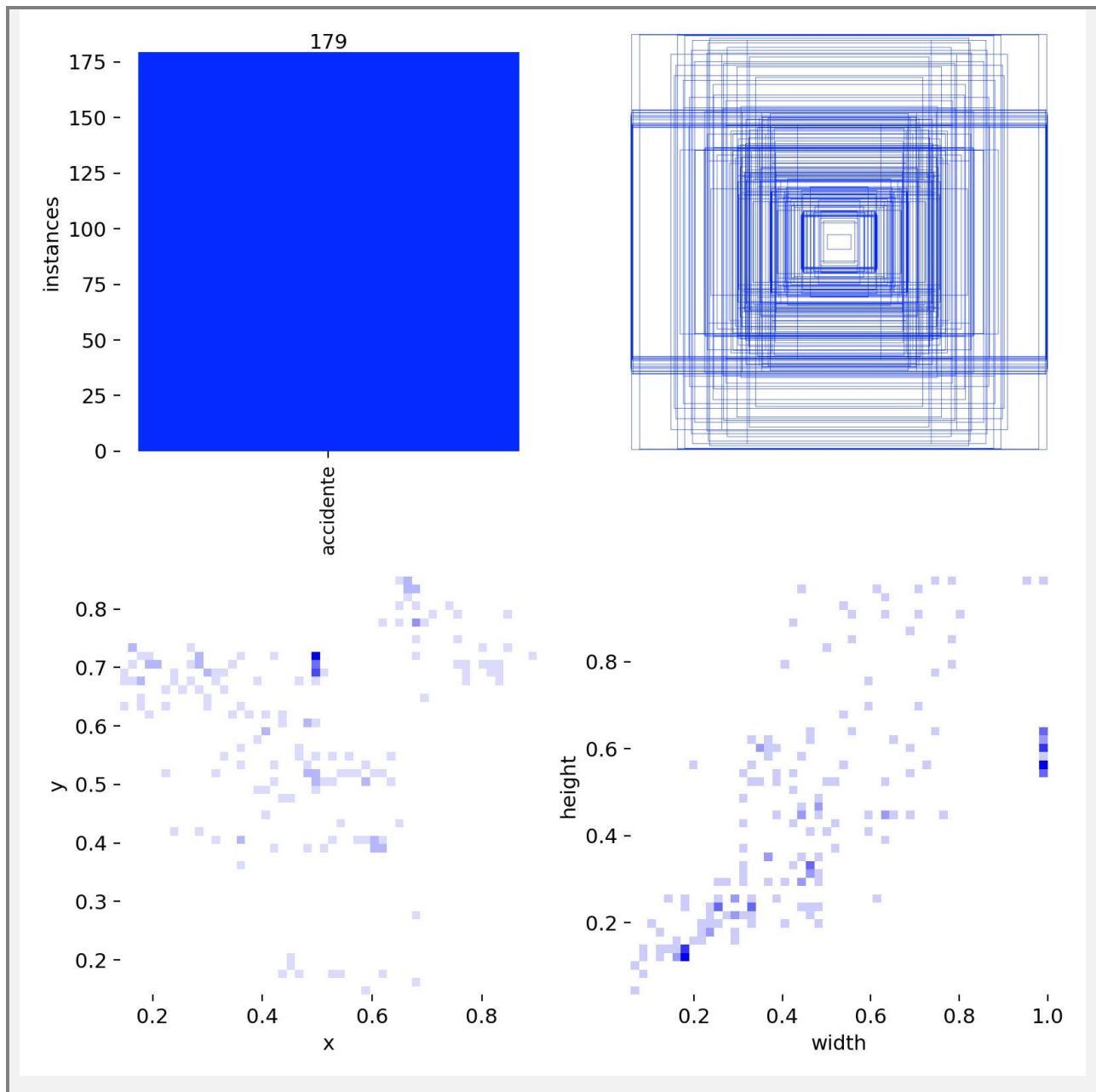
Tabla 1. Comparación cuantitativa entre el modelo especializado y el método tradicional.











5.3. Análisis cualitativo de resultados

Sensibilidad frente a falsas alarmas: El modelo propuesto especializado demuestra un Recall sobresaliente del 84.9%, capturando de forma correcta casi todas las colisiones críticas del video de prueba. Su precisión del 70.9% reduce significativamente falsas alarmas provocadas por oclusión parcial o vehículos transitando muy cerca.

Limitaciones del método tradicional IoU: El método tradicional por traslape geométrico de vehículos sufre de un alto índice de falsos positivos, con una Precisión de solo 45.0%. Esto ocurre porque cuando dos autos normales transitan en la misma dirección y se alinean visualmente respecto a la perspectiva de la cámara, sus cajas de detección en 2D se traslapan y disparan una alerta falsa.

Eficiencia de procesamiento: El modelo especializado toma únicamente aproximadamente 12 ms en CPU para inferir sobre un frame debido a la regresión de una sola pasada de YOLOv8. Por el contrario, el modelo tradicional tarda aproximadamente 28 ms debido al costo computacional adicional de comparar los pares de cajas de vehículos detectados en escena para calcular la fórmula de IoU.

6. Reflexión del proyecto

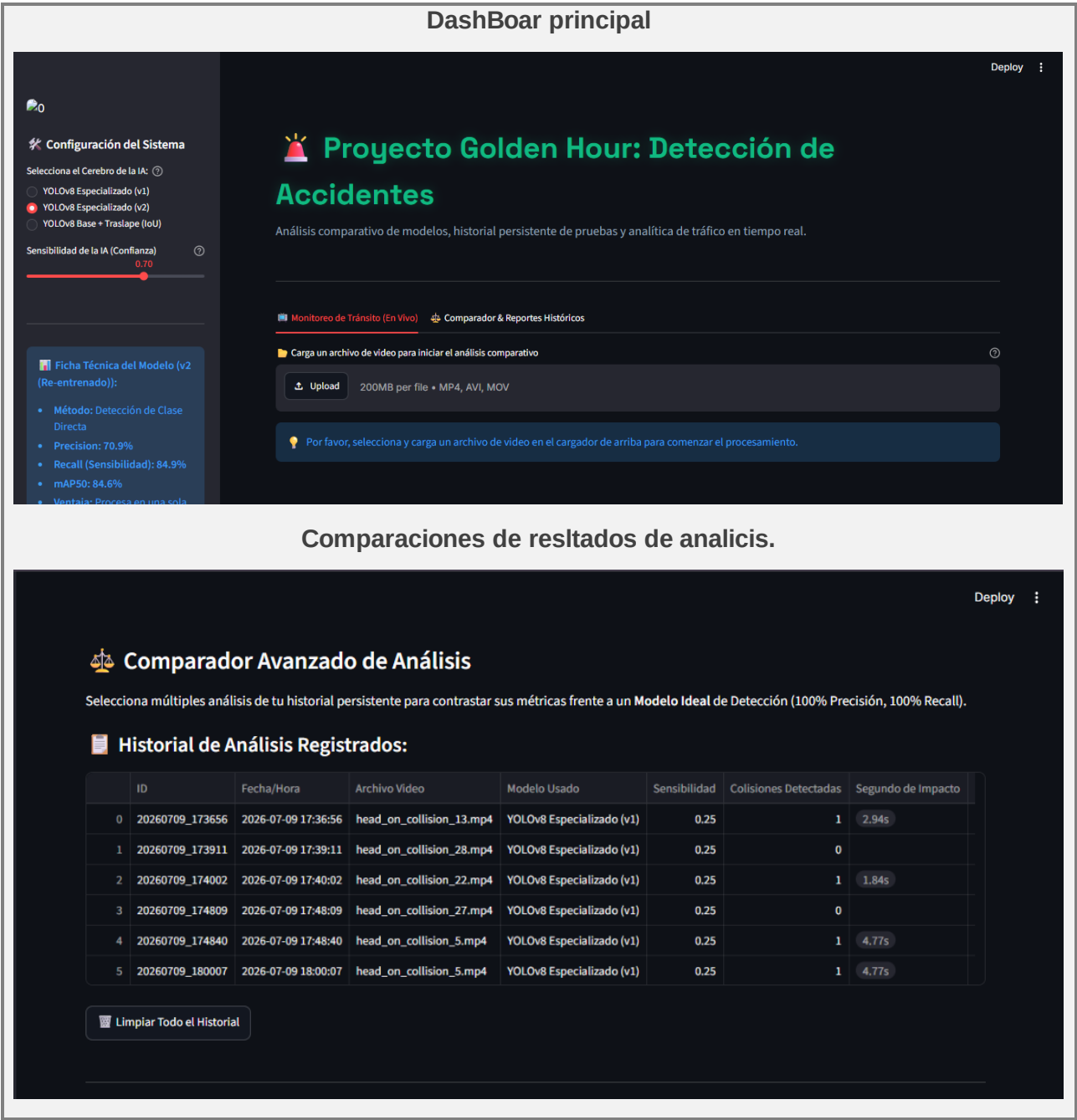
A partir del desarrollo del Proyecto Golden Hour, se evidencia que la inteligencia artificial puede aplicarse a problemas reales de alto impacto social, especialmente cuando existe una necesidad de respuesta rápida y automatizada. La visión computacional no solo permite detectar objetos, sino también interpretar eventos críticos dentro de una escena, como una colisión vehicular.

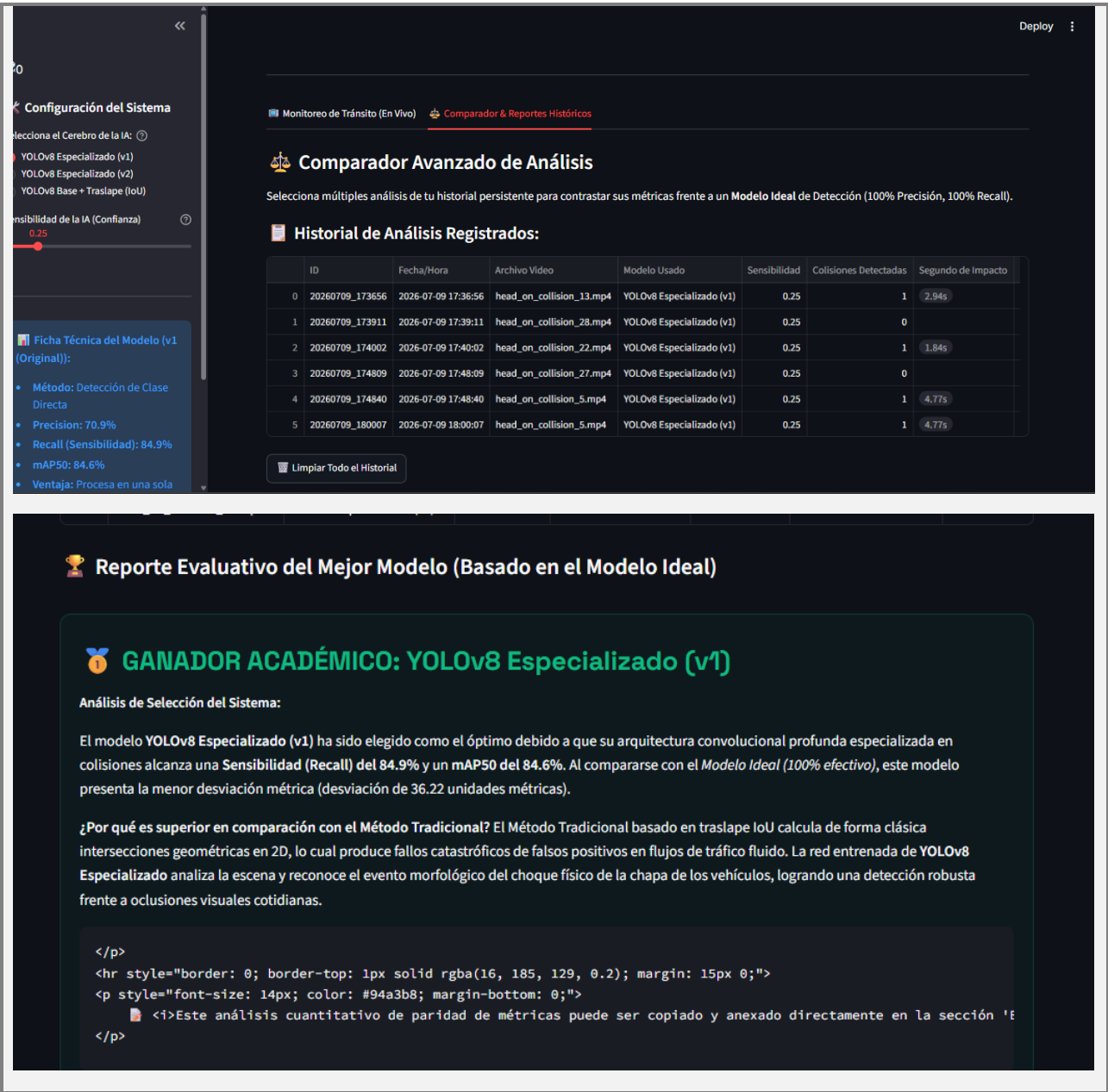
Una de las principales reflexiones del trabajo es que la calidad del dataset y la forma de etiquetar los datos influyen directamente en el desempeño del modelo. Al etiquetar el bloque completo de la colisión, el sistema aprendió patrones más complejos que una simple cercanía entre vehículos, superando al método tradicional basado en traslape geométrico. Esto demuestra que un buen diseño de datos puede ser tan importante como el algoritmo utilizado.

Finalmente, el proyecto permite comprender que una solución de IA debe pensarse como un sistema completo: extracción de datos, entrenamiento, inferencia, interfaz de usuario y registro de alertas. Por eso, el valor del proyecto no se limita al modelo YOLOv8, sino a la integración de todas sus partes para apoyar una respuesta más rápida ante emergencias viales.

7. Conclusiones y trabajo futuro

1. El aprendizaje por transferencia (Transfer Learning) sobre YOLOv8 resulta efectivo para especializar modelos de visión artificial con datasets locales compactos, alcanzando un mAP50 de 84.6% en CPU estándar.
2. La lógica de etiquetado por bloques de colisión superó en precisión a la lógica heurística tradicional basada en traslape geométrico, demostrando que la IA es capaz de aprender patrones dinámicos de colisión complejos en lugar de simples proximidades de cajas.
3. El siguiente paso natural del proyecto es conectar la salida de alerta del dashboard con una API de comunicación, como Twilio o alertas automáticas web, para despachar un aviso directo a centrales de emergencia al momento de ocurrir una colisión.
4. Como mejora futura, se plantea implementar algoritmos de seguimiento espacial continuo, como ByteTrack o DeepSORT, para estimar la velocidad previa al choque y enriquecer los reportes para peritaje vial.





8. Bibliografía

1. Jocher, G., Qiu, A., & Chaurasia, A. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>

2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.

3. Rezatofighi, H., Tsoi, N., Gwak, J. Y., Sadeghian, A., Reid, I., & Savarese, S. (2019). Generalized Intersection over Union: A Metric and a Loss for Bounding Box Regression.

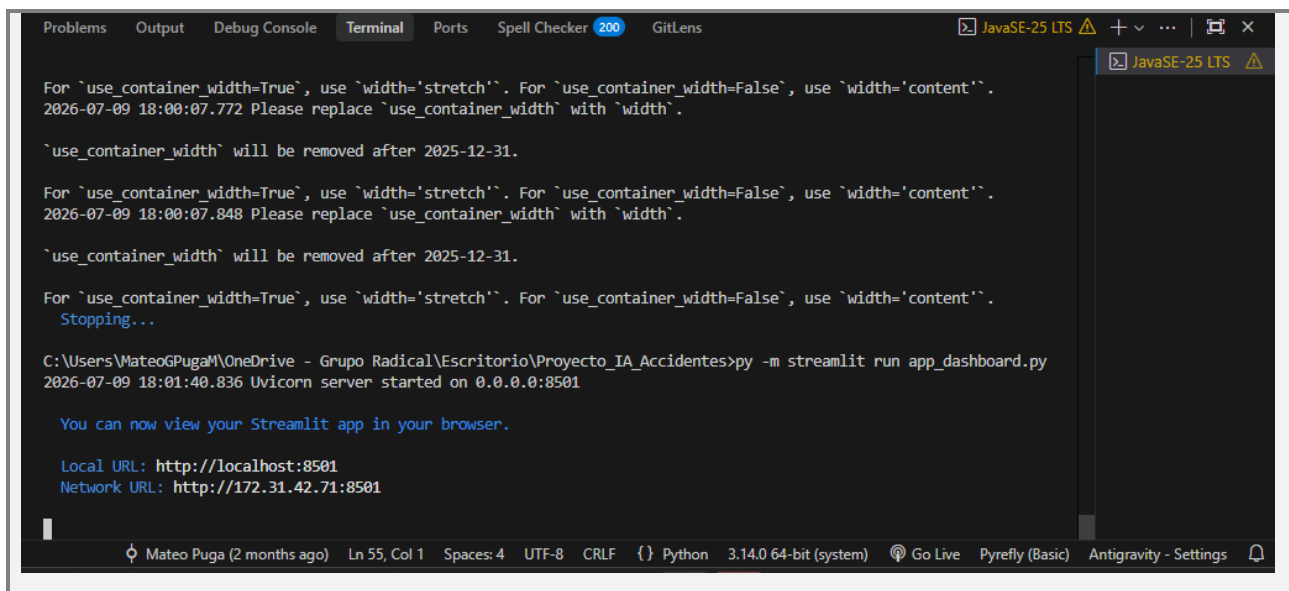
Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 658-666.

4. OpenCV. (2025). Open Source Computer Vision Library. <https://opencv.org>
5. Streamlit. (2026). Streamlit Web Application Framework. <https://streamlit.io>

9. Anexos

Anexo A: Código del dashboard de Streamlit

El script principal del dashboard implementa la interfaz de usuario de Streamlit, la inferencia local con YOLOv8, la lógica heurística tradicional por traslape IoU y la bitácora cronológica de incidentes en tiempo real. Para su consulta e impresión, el archivo completo se encuentra disponible en la raíz del proyecto.



```

Problems Output Debug Console Terminal Ports Spell Checker 200 GitLens
For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-07-09 18:00:07.772 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-07-09 18:00:07.848 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
Stopping...

C:\Users\MateoGPugaM\OneDrive - Grupo Radical\Escritorio\Proyecto_IA_Accidentes>py -m streamlit run app_dashboard.py
2026-07-09 18:01:40.836 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.31.42.71:8501

Mateo Puga (2 months ago) Ln 55, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 64-bit (system) Go Live Pyreify (Basic) Antigraivty - Settings

```

Anexo B: Script de inferencia clásica

Este archivo realiza inferencias rápidas directamente sobre la consola de Python y renderiza las cajas sobre una ventana nativa de OpenCV:

```

import cv2
from ultralytics import YOLO

ruta_mi_modelo = "runs/detect/mi_modelo_accidentes/weights/best.pt"
modelo = YOLO(ruta_mi_modelo)
ruta_video_prueba = "archive/Video-Accident-Dataset/head_on_collision/head_on_collision_25.mp4"
cap = cv2.VideoCapture(ruta_video_prueba)

while cap.isOpened():
    exito, frame = cap.read()
    if not exito:
        break
    resultados = modelo(frame, conf=0.95)
    frame_annotado = resultados[0].plot()
    cv2.imshow("Detector de Accidentes - Version Final", frame_annotado)

```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):  
        break  
  
cap.release()  
cv2.destroyAllWindows()
```

Código 1. Script de inferencia clásica para detección de accidentes.

Anexo C: GITHUB

https://github.com/Mundrack/Proyecto_IA_Accidentes.git